

```

import io
import zipfile
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from google.colab import files

# =====
# 1. UPLOAD THE DATASET DIRECTLY TO COLAB
# =====
print("Please click 'Choose Files' below and select your 'archive.zip' file:")
uploaded = files.upload()

# Get the filename from the uploaded dict keys
zip_filename = list(uploaded.keys())[0]
print(f"\nSuccessfully uploaded: {zip_filename}")

# =====
# 2. EXTRACT AND LOAD THE DATASET
# =====
# Open the uploaded zip file directly from memory and read the CSV
with zipfile.ZipFile(io.BytesIO(uploaded[zip_filename]), 'r') as zip_ref:
    # Look for the CSV file name inside the zip
    csv_file_name = [f for f in zip_ref.namelist() if f.endswith('.csv')][0]
    with zip_ref.open(csv_file_name) as csv_file:
        df = pd.read_csv(csv_file)

print(f"Initial Dataset Loaded! Shape: {df.shape}\n")

# =====
# 3. DATA CLEANING & PREPROCESSING
# =====
print("--- Starting Data Cleaning ---")

# Step A: Handle Outliers (Age & Employment)
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] < (df['person_age'] - 14)]

# Step B: Handle Missing Values (Imputation)
df['person_emp_length'] = df['person_emp_length'].fillna(df['person_emp_length'].median())
df['loan_int_rate'] = df.groupby('loan_grade')['loan_int_rate'].transform(lambda x: x.fillna(x.median()))

# Step C: Remove Duplicates
df.drop_duplicates(inplace=True)
print(f"Cleaned Dataset Shape: {df.shape}\n")

# =====
# 4. CATEGORICAL ENCODING
# =====
# One-hot encode nominal categories
df_encoded = pd.get_dummies(df, columns=['person_home_ownership', 'loan_intent'], drop_first=True)

# Ordinal encode 'loan_grade' (A=1, B=2, ..., G=7)
grade_manning = {'A': 1, 'B': 2, 'C': 3, 'D': 4, 'E': 5, 'F': 6, 'G': 7}

```

```

df_encoded['loan_grade_encoded'] = df_encoded['loan_grade'].map(grade_mapping)

# Convert binary indicator to numeric flag (Y=1, N=0)
df_encoded['cb_person_default_on_file'] = df_encoded['cb_person_default_on_file'].map({'Y': 1, 'N': 0})

# =====
# 5. DEEP-DIVE VISUALIZATIONS
# =====
print("--- Generating Visualizations ---")
sns.set_theme(style="whitegrid")
plt.figure(figsize=(15, 10))

# 1. Correlation Matrix Heatmap
plt.subplot(2, 2, 1)
numeric_cols = df_encoded.select_dtypes(include=[np.number]).columns
corr_matrix = df_encoded[numeric_cols].corr()
sns.heatmap(corr_matrix[['loan_status']].sort_values(by='loan_status', ascending=False),
            annot=True, cmap='coolwarm', vmin=-1, vmax=1, cbar=False)
plt.title("Drivers Correlated with Loan Default")

# 2. Risk Segment by Loan Intent
plt.subplot(2, 2, 2)
sns.barplot(data=df, x='loan_intent', y='loan_status', errorbar=None, palette='viridis')
plt.xticks(rotation=30)
plt.ylabel("Default Rate")
plt.title("Default Rates by Loan Intent")

# 3. Default Rates by Home Ownership
plt.subplot(2, 2, 3)
sns.barplot(data=df, x='person_home_ownership', y='loan_status', errorbar=None, palette='magma')
plt.ylabel("Default Rate")
plt.title("Default Rates by Home Ownership")

# 4. Default Risk Scale Across Underwriting Grades
plt.subplot(2, 2, 4)
order_grades = sorted(df['loan_grade'].unique())
sns.pointplot(data=df, x='loan_grade', y='loan_status', order=order_grades, color='crimson')
plt.ylabel("Default Rate")
plt.title("Default Escalation Across Loan Grades")

plt.tight_layout()
plt.show()

# =====
# 6. PRINT TEXT INSIGHTS
# =====
print("\n--- Summary Insights ---")
print("\nTop 3 Loan Intent categories with highest default rates:")
print(df.groupby('loan_intent')['loan_status'].mean().sort_values(ascending=False).head(3))

print("\nDefault rate based on Home Ownership status:")
print(df.groupby('person_home_ownership')['loan_status'].mean().sort_values(ascending=False))

```


Please click 'Choose Files' below and select your 'archive.zip' file:
 archive.zip
archive.zip(application/x-zip-compressed) - 376518 bytes, last modified: 16/06/2026 - 100% done
Saving archive.zip to archive.zip

Successfully uploaded: archive.zip
Initial Dataset Loaded! Shape: (32581, 12)

--- Starting Data Cleaning ---
Cleaned Dataset Shape: (31522, 12)

--- Generating Visualizations ---
/tmp/ipykernel_978/4199811821.py:78: FutureWarning:

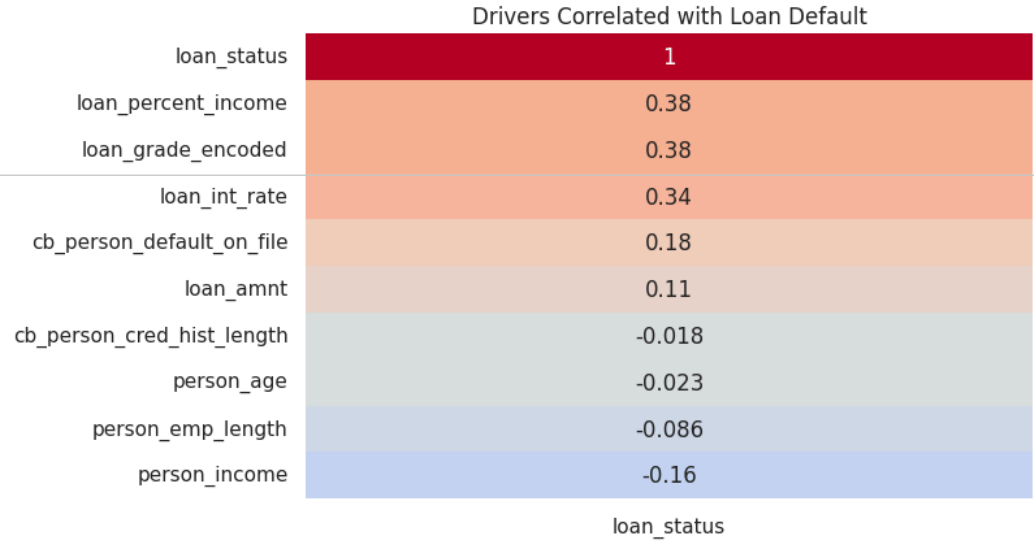
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='loan_intent', y='loan_status', errorbar=None, palette='viridis')
```

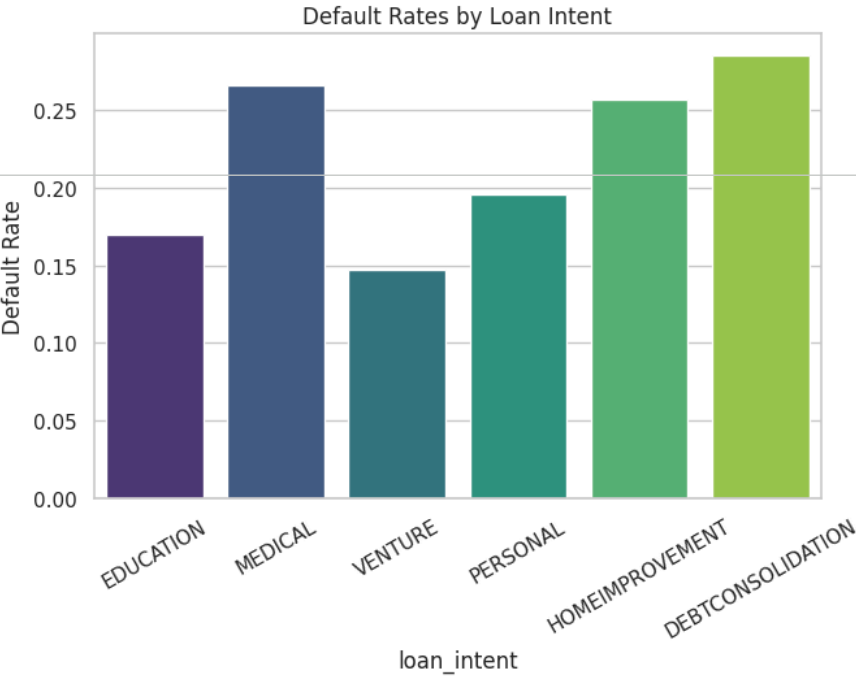
/tmp/ipykernel_978/4199811821.py:85: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='person_home_ownership', y='loan_status', errorbar=None, palette='magma')
```



Default Rates by Home Ownership



Default Escalation Across Loan Grades